

Threat Intelligence Integration with OpenCTI

Connecting OpenCTI Threat Intelligence to a Wazuh SIEM Detection Pipeline

Prepared by

Makinde Boluwatife

1. Executive Summary

This project added OpenCTI threat intelligence to an existing Wazuh SIEM lab. The main goal was to use external threat intelligence to add useful context to security alerts and improve detection.

The project was completed in four phases: deploying OpenCTI, connecting threat-intelligence feeds, building the OpenCTI-to-Wazuh enrichment workflow, and improving a Wazuh detection rule using CTI and MITRE ATT&CK context.

For Phase 4, the existing Rule 100400 detected scheduled-task creation at Level 8. I created Rule 100600 to look for the same activity when PowerShell was the parent process. The new rule raised the alert to Level 12 and added MITRE ATT&CK technique T1059.001 for PowerShell. Both rules were tested with controlled activity on the Windows endpoint.

2. Objectives

- Deploy OpenCTI and connect it to external threat-intelligence feeds.
- Connect OpenCTI with the existing Wazuh SIEM using a CDB indicator list and a custom enrichment script
- Test the OpenCTI enrichment workflow with a controlled Wazuh alert and use real Wazuh telemetry where applicable.
- Use CTI and MITRE ATT&CK context to improve an existing Wazuh detection rule.

3. Architecture

The lab used a Windows endpoint with Wazuh Agent and Sysmon, a Wazuh Manager and Dashboard, and OpenCTI. Together, these components provided endpoint telemetry, detection, threat-intelligence enrichment, MITRE ATT&CK context, and analyst investigation.

Capstone Lab Architecture

CTI-Informed Threat Detection with OpenCTI and Wazuh

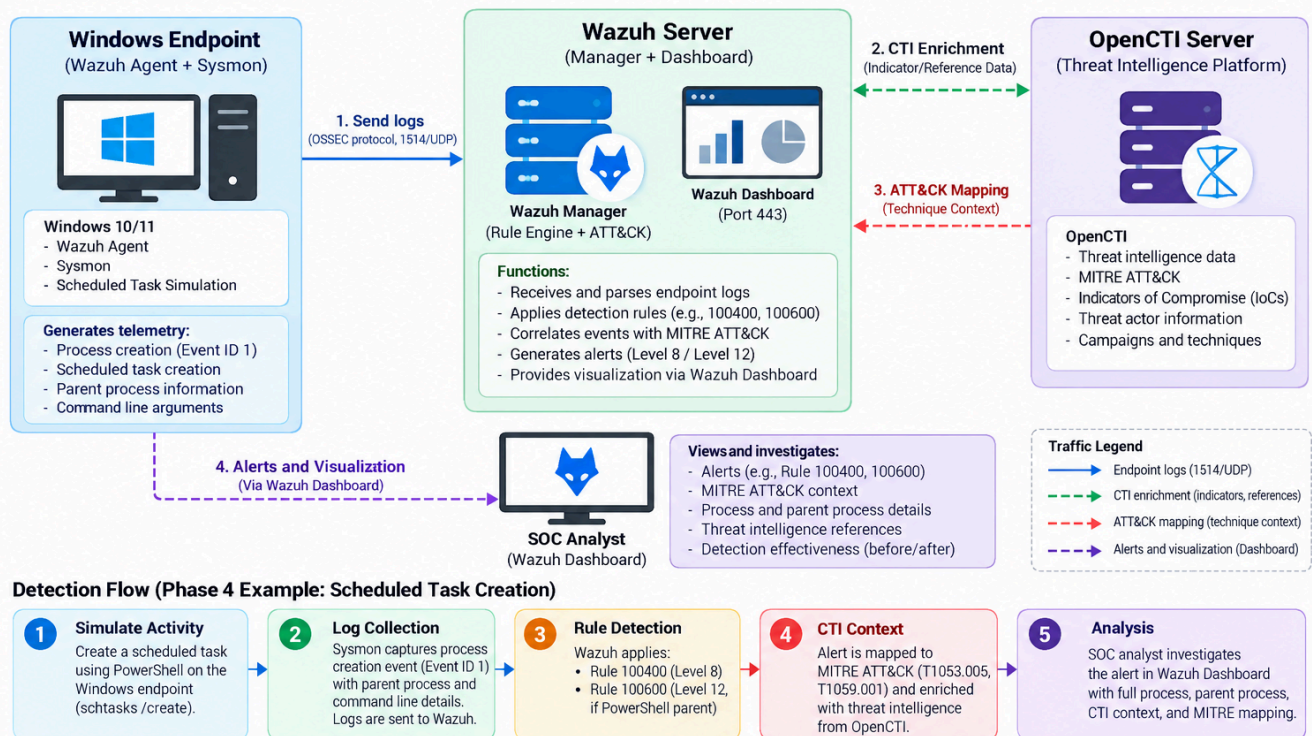


Figure 1. CTI-informed SOC architecture used for the capstone project.

Two integration paths connect the platforms

- Batch export (CDB list): Indicators from OpenCTI were exported to the Wazuh CDB list opencti-ips. Wazuh can use this list to check network events against known indicators.
- Live enrichment lookup: When a Wazuh rule fires, wazuh-integrator runs a custom script. The script queries the OpenCTI API and writes the returned threat context to opencti_enriched_alerts.log.

The enrichment workflow was successfully tested with a controlled Wazuh alert. The script queried OpenCTI for 156.229.165.166, found the related threat-intelligence record, and wrote the enrichment result to the log. However, the live custom Wazuh IP-match alert did not fire because of a documented Sysmon RuleName evaluation limitation. I did not claim a live IOC match where one was not observed.

4. Feed Selection

OpenCTI was configured with several external import connectors to source threat intelligence automatically

rather than relying on manually entered indicators. At the time of writing, the platform had 12 active connectors, including:

- **MITRE ATT&CK:** provides the tactics, techniques, and sub-techniques used throughout this project to map detections to a standard adversary behavior framework.
- **ThreatFox (Abuse.ch):** supplies actively tracked malicious IP and domain indicators with type/family tagging (Coinminer, botnet_cc, xmrig, etc.)
- **OpenCTI Datasets and AlienVault (OTX):** provide broader coverage of reports, malware, and intrusion sets used for general platform population and analyst context.

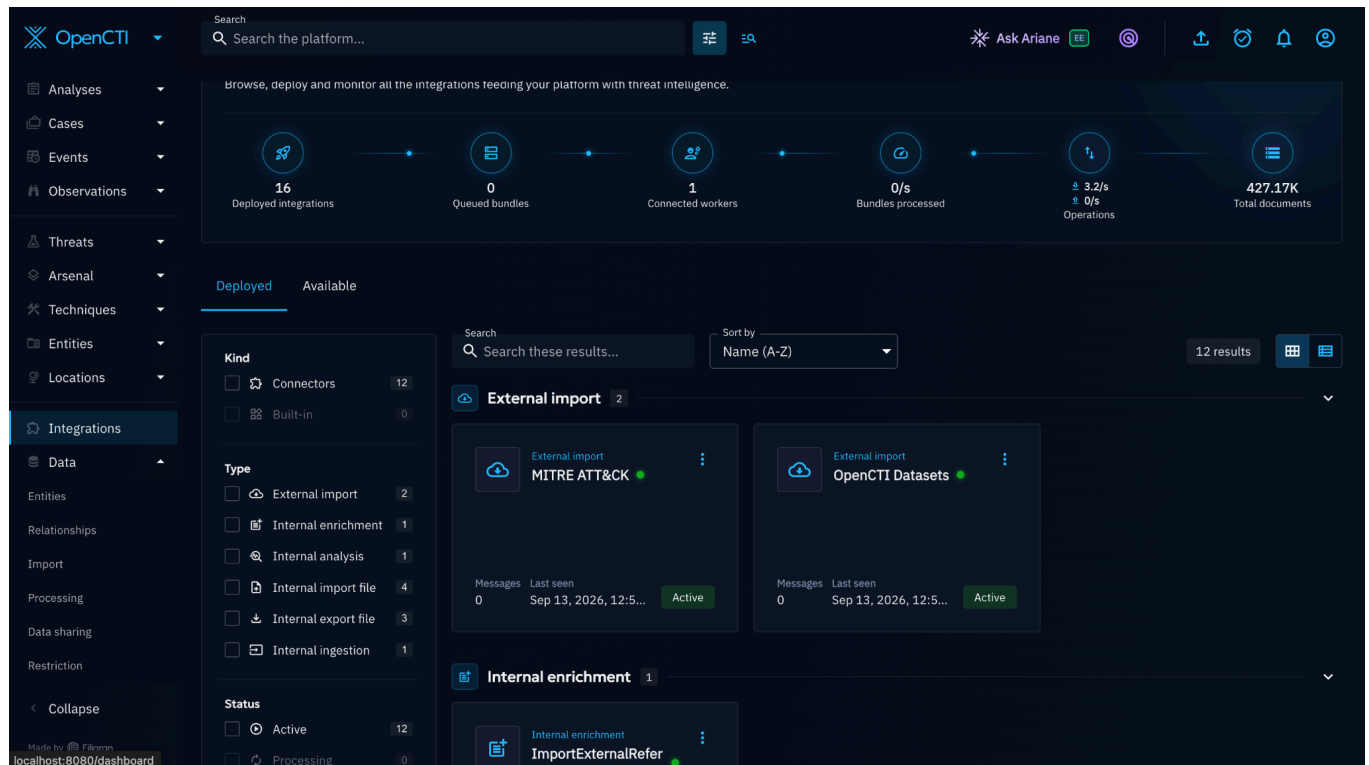


Figure 2. OpenCTI Integrations view showing active external-import connectors, including MITRE ATT&CK and OpenCTI Datasets, alongside internal enrichment, import, and export connectors.

The result of this feed selection is a platform genuinely populated with live data rather than a sparse test instance. The platform held 193 intrusion sets, 935 malware entities, 1.78K indicators, and 13 reports, with Kimsuky, APT28, and Lazarus Group as the most active tracked threats over the prior three months.

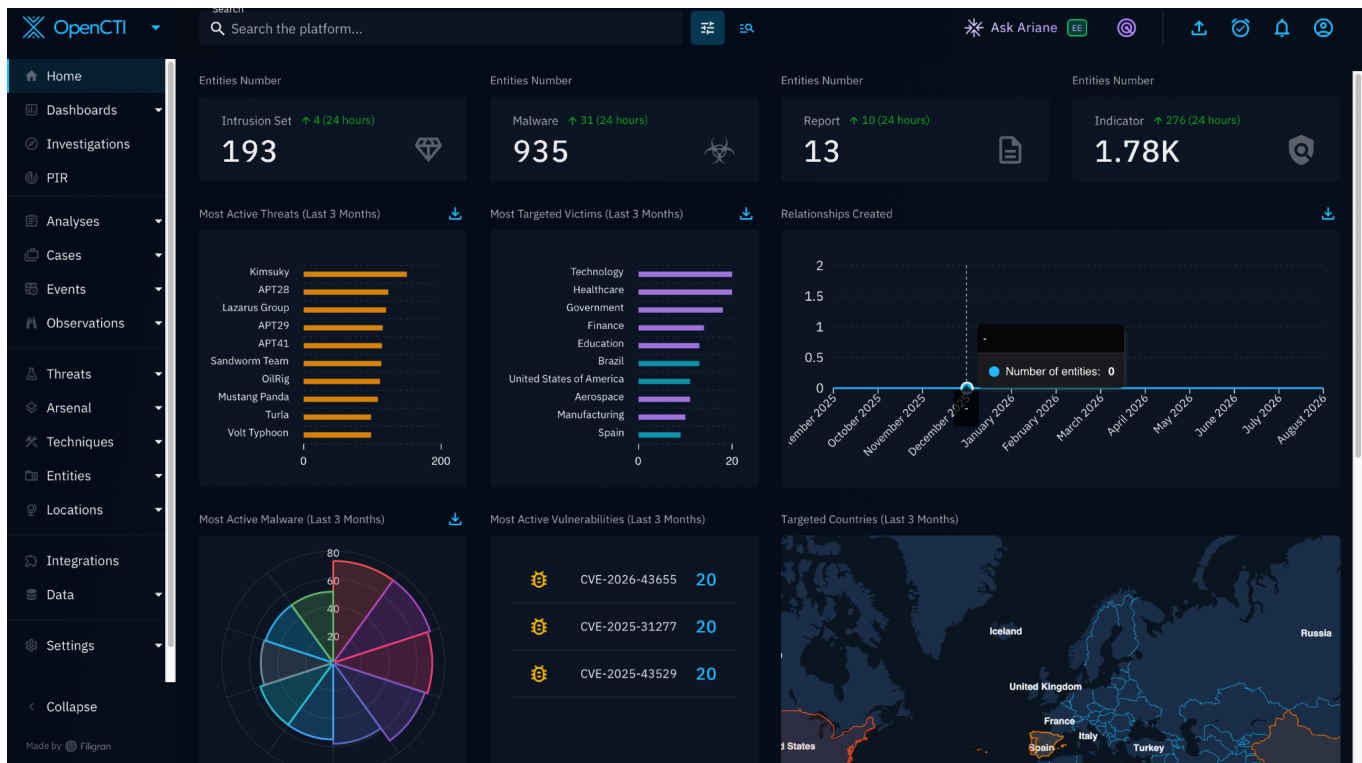


Figure 3. OpenCTI home dashboard confirming a live, populated platform: entity counts, most active threats and malware, most targeted sectors/countries, and tracked CVEs.

The specific indicator central to this project's enrichment work, the IPv4 address 156.229.165.166 used validation is sourced from ThreatFox and tagged Coinminer / botnet_cc / xmrig, with a confidence score of 50/100:

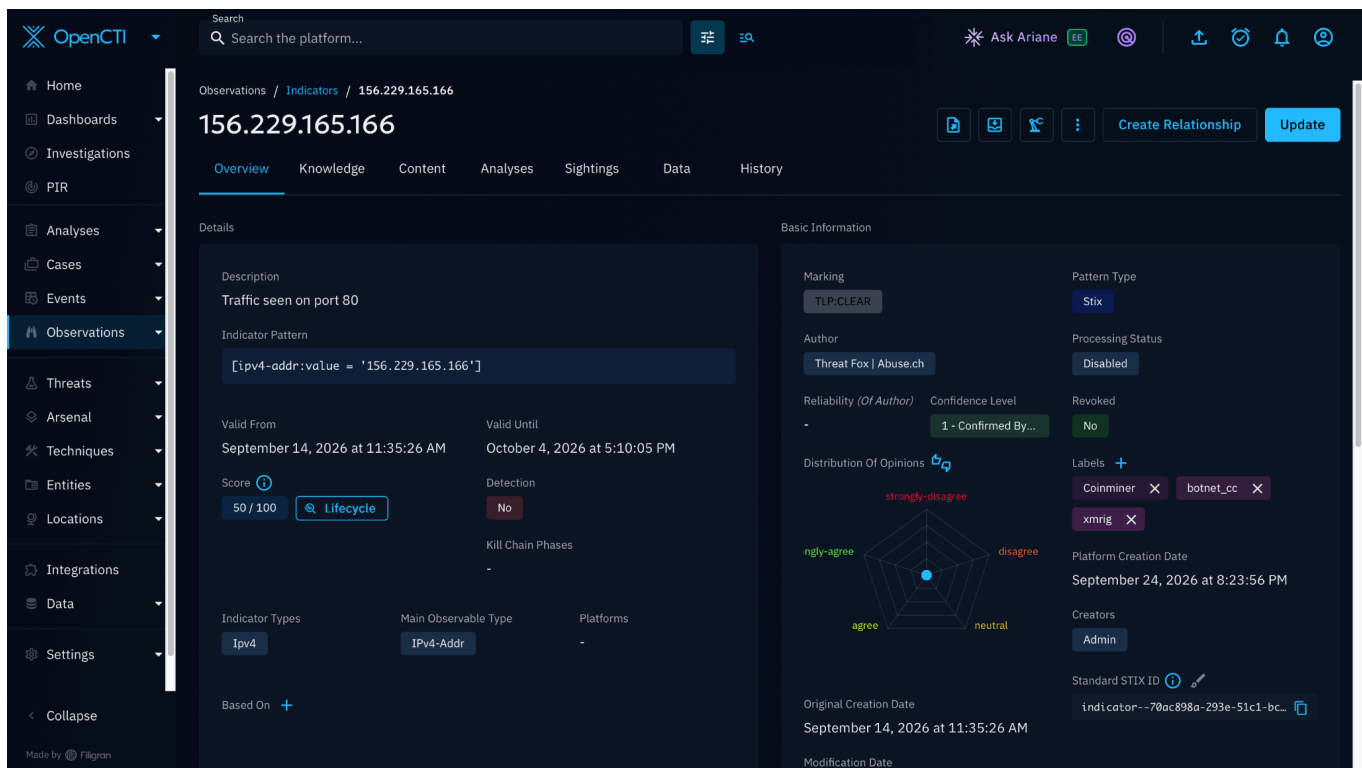


Figure 4. OpenCTI Indicator detail for 156.229.165.166: STIX pattern, ThreatFox authorship, TLP:CLEAR marking, and Coinminer/botnet_cc/xmrig labels — the exact record returned by the Phase 3 enrichment script.

5. Enrichment Pipeline: Design and Obstacles

5.1 Design

The enrichment workflow used a custom script instead of OpenCTI Live Streams or TAXII. The script calls the OpenCTI REST API when Wazuh sends an alert for enrichment. This approach was suitable for the lab because it avoided running a permanent stream consumer.

5.2 Obstacle: host resource contention

Running OpenCTI and several VirtualBox VMs on an 8 GB computer caused resource pressure, especially during Elasticsearch startup. To keep the lab stable, I stopped non-essential OpenCTI containers and avoided running all heavy services at the same time.

5.3 Obstacle: live rule-evaluation gap for unnamed Sysmon rules (documented limitation)

- The built-in Sysmon Event ID 3 rule chain was confirmed to receive the live event.
- wazuh-logtest confirmed that the custom CDB lookup and IP match were valid.
- The problem was reproduced with the malicious IP and with 8.8.8.8, so it was not caused by the indicator itself.
- A zero-condition diagnostic rule also did not fire on the affected event pattern.
- Rule loading, CDB configuration, decoder fields, queues, and rate limiting were checked during troubleshooting.
- The affected events contained Sysmon RuleName: - while other Sysmon events with a named rule could trigger custom rules.
- The issue was therefore recorded as a scoped live analysis rule-evaluation limitation for this event pattern, not as a failure of OpenCTI or the CDB list.
- The custom enrichment workflow was still independently demonstrated with the controlled alert test.

The main lesson was to separate four stages during troubleshooting: receiving the event, decoding it, loading the rule, and generating the alert. This made it possible to identify the limitation without claiming a result that was not observed.

6. Applied Use Case: Before / After

6.1 Baseline: Rule 100400

Rule 100400 (Level 8) detects scheduled-task creation with schtasks.exe and maps the event to MITRE ATT&CK T1053.005. It was tested successfully with a controlled scheduled-task simulation.

The Sysmon event also contained parentImage information. The original rule did not use this field, so it could not distinguish a scheduled task created by PowerShell from other scheduled-task activity.

6.2 Improvement: Rule 100600

Rule 100600 (Level 12) was added without changing Rule 100400. It looks for a scheduled task created by schtasks.exe when PowerShell or pwsh.exe is the parent process. It maps the alert to T1053.005 and T1059.001.

```
<rule id="100600" level="12">
  <if_sid>100400</if_sid>
  <field name="win.eventdata.parentImage" type="pcre2">(?!\\)(powershell|pwsh)\\.exe$</field>
<description>CTI-informed detection: Scheduled task created by PowerShell - elevated persistence
risk</description>
  <mitre>
    <id>T1053.005</id>
    <id>T1059.001</id>
  </mitre>
  <group>persistence,scheduled_task,powershell,cti_informed,</group>
</rule>
```

The improvement uses existing Sysmon data. The parent process was already being collected; the new rule simply evaluates that field and adds ATT&CK context.

Both rules were tested with controlled scheduled-task creation on the Windows endpoint. Atomic Red Team was not used for the final test. A manual scheduled-task simulation was used instead, which is acceptable under the project guide because it allows simulated traffic or an Atomic Red Team test.

6.3 Before / After Comparison

Rule level	8	12
Fired (timestamp)	2026-09-30 19:13:39	2026-09-30 19:41:02
Description	Scheduled task created via schtasks.exe - possible persistence mechanism	CTI-informed detection: Scheduled task created by PowerShell - elevated persistence risk
MITRE ATT&CK technique(s)	T1053.005	T1053.005 + T1059.001
Email notification	No	Yes
Test task name	SOC-Phase4-Baseline	SOC-Phase4-Improved
Parent process condition	Not evaluated by rule (present in raw Sysmon data but unused)	powershell.exe explicitly required

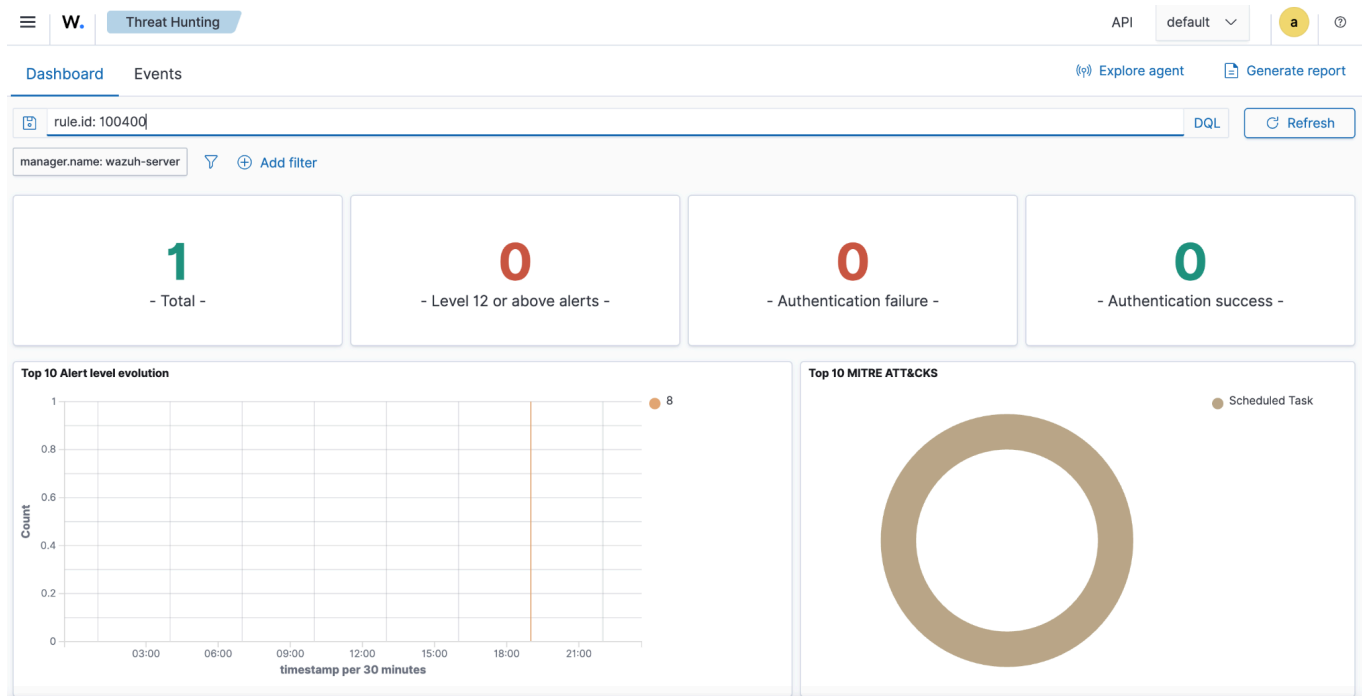


Figure 5a. Wazuh Threat Hunting dashboard for rule.id:100400 - 1 alert, Level 8, MITRE mapped to Scheduled Task only.

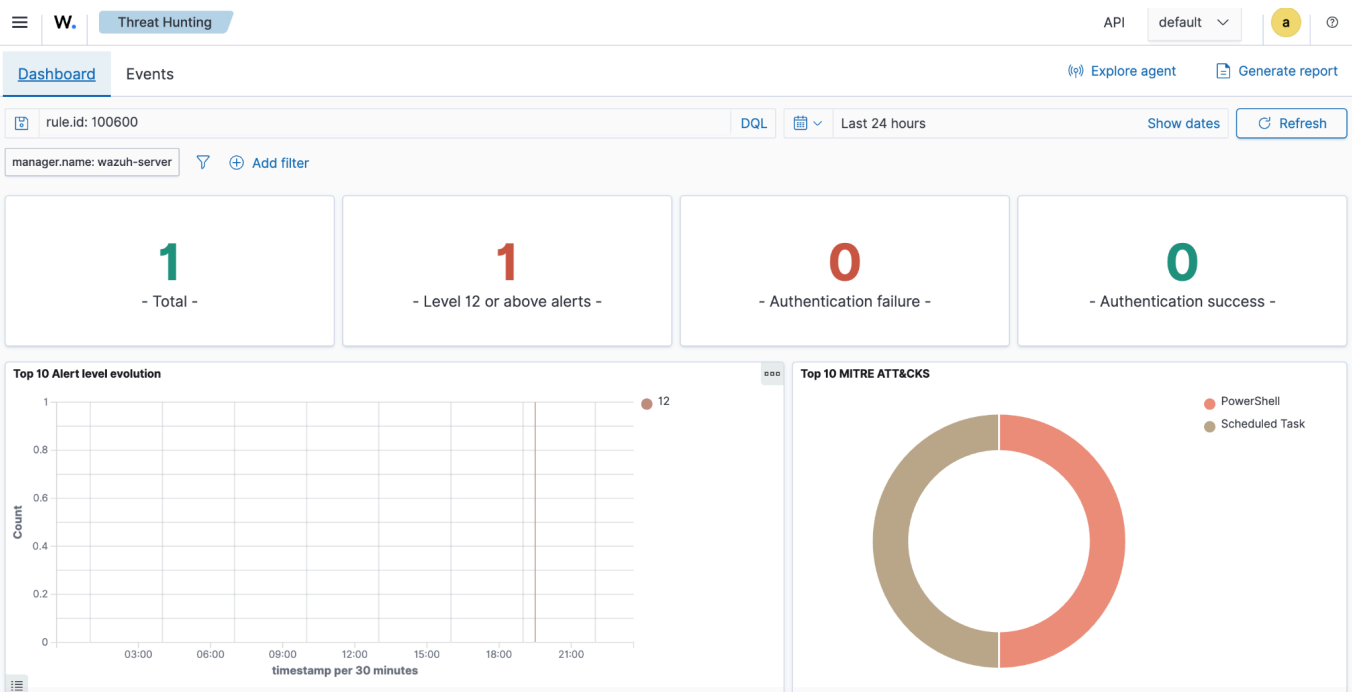


Figure 5b. Wazuh Threat Hunting dashboard for rule.id:100600 - 1 alert, Level 12 (flagged as Level-12-or-above), MITRE mapped to both Scheduled Task and PowerShell.

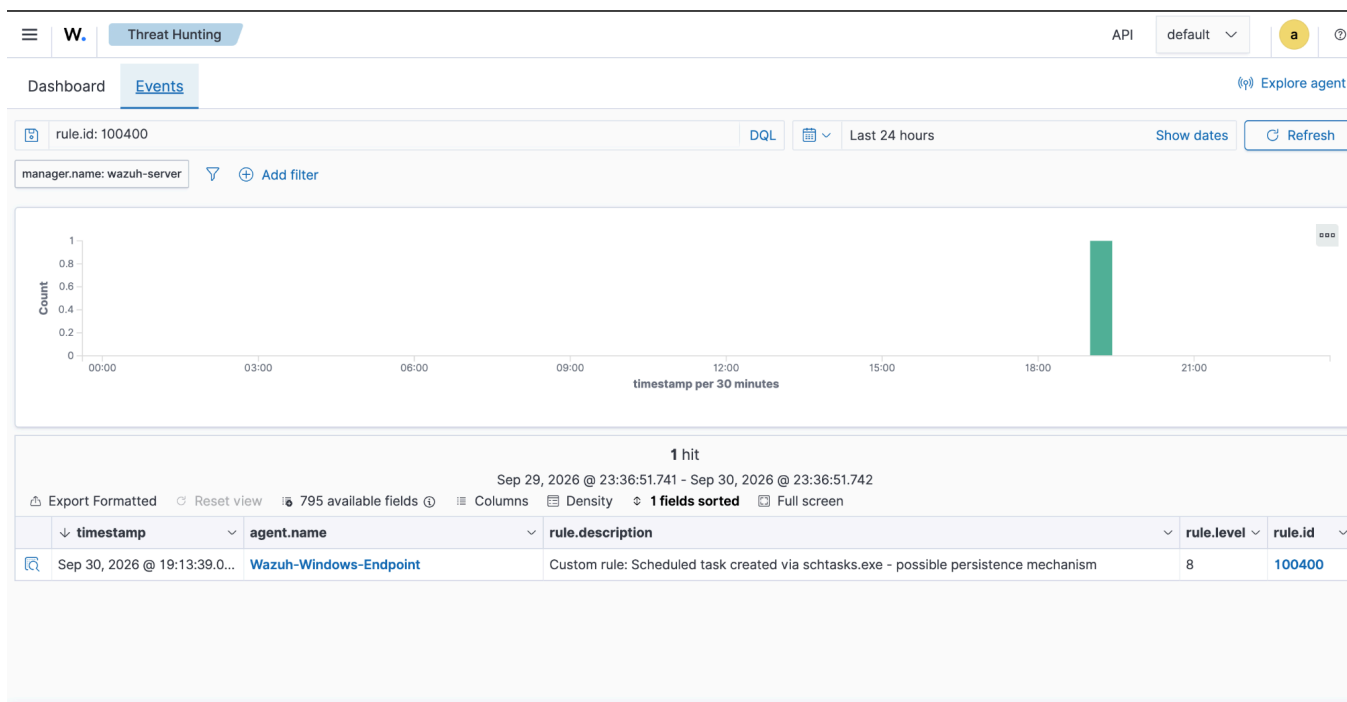


Figure 6a. Baseline alert (Rule 100400): "Custom rule: Scheduled task created via schtasks.exe - possible persistence mechanism", Level 8, fired 2026-09-30 19:13:39 on Wazuh-Windows-Endpoint.

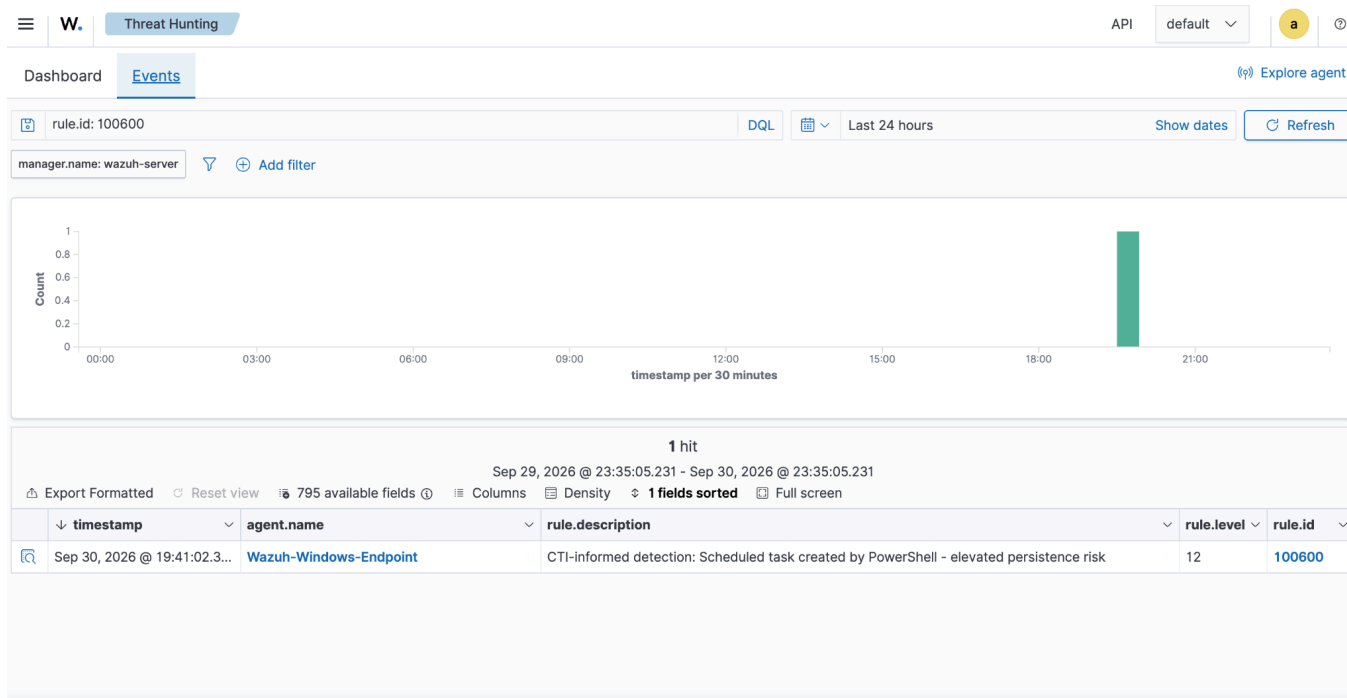


Figure 6b. Improved alert (Rule 100600): "CTI-informed detection: Scheduled task created by PowerShell - elevated persistence risk", Level 12, fired 2026-09-30 19:41:02 on Wazuh-Windows-Endpoint.

Rule 100600 did not require new telemetry. The original Sysmon event already contained ParentImage=powershell.exe. The improvement came from using information that was already being collected but was not used by the original rule. This shows how reviewing existing telemetry can improve detection without adding new tools.

7. Lessons Learned

- Review existing telemetry before adding new data sources.
- Choose the integration method based on the lab's needs. A CDB list is useful for indicator matching, while API enrichment provides more context when an alert is available.
- Manage resources carefully when running OpenCTI and virtual machines on a small computer.
- Treat platform limitations as findings when testing shows a repeatable problem.
- Use wazuh-logtest to check rule logic against real event data and separate rule problems from live pipeline problems.

8. Conclusion

This project connected OpenCTI threat intelligence with a Wazuh SIEM lab and showed how CTI can support security detection and investigation.

The OpenCTI enrichment workflow was successfully demonstrated with a controlled Wazuh alert and a real threat-intelligence indicator. A live custom IOC alerting limitation was also identified, investigated, and documented.

CTI and MITRE ATT&CK context helped improve a scheduled-task detection from Level 8 to Level 12 by adding PowerShell parent-process context. The project therefore demonstrated a practical SOC skill: using threat intelligence and existing telemetry to make detections more useful for analysts.

Appendix A - Key Alert Fields (alerts.json)

Extracted fields for both test alerts referenced in Section 6, as recorded from Wazuh's alerts.json on the manager during Phase 4 validation. Presented as extracted key fields rather than the full raw JSON document.

A.1 Rule 100400

rule.id	100400
rule.level	8
timestamp	2026-09-30 18:13:39 (manager local time; shown as 19:13:39 in the Wazuh dashboard's display timezone)
agent.name	Wazuh-Windows-Endpoint (agent ID 004)
rule.description	Scheduled task created via schtasks.exe - possible persistence mechanism
data.win.eventdata.taskName	SOC-Phase4-Baseline
email notification (mail)	false

A.2 Rule 100600

rule.id	100600
rule.level	12
timestamp	2026-09-30 18:41:02 (manager local time; shown as 19:41:02 in the Wazuh dashboard's display timezone)
agent.name	Wazuh-Windows-Endpoint (agent ID 004)
rule.description	CTI-informed detection: Scheduled task created by PowerShell - elevated persistence risk
data.win.eventdata.taskName	SOC-Phase4-Improved
data.win.eventdata.parentImage	powershell.exe
rule.mitre.id	T1053.005, T1059.001